

✓ oops in python

1. class
2. object
3. Encapsulation
4. Inheritance
5. polymorphism
6. abstraction

doc string

written in tripple quotes

```
""" """
```

dictionary

it stores data in key:value pair format

```
student={  
    "name":"akhil",  
    "age":20,  
    "place":"pulivendula"  
}
```

dictionary powers:-

- ~ mutable
- ~ no duplicate keys
- ~ ordered
- ~ keys must be immutable

```
student={  
    "name":"akhil",  
    "age":20,  
    "place":"pulivendula"  
}  
print(student)
```

```
{'name': 'akhil', 'age': 20, 'place': 'pulivendula'}
```

```
student={  
    "name":"akhil",  
    "age":20,  
    "place":"pulivendula"  
}  
print(student["name"])
```

akhil

```
student={  
    "name":"akhil",  
    "age":20,  
    "place":"pulivendula"  
}  
print(f"Name: {student['name']}, Age: {student['age']}")
```

akhil
Name: akhil, Age: 20

```
student={  
    "name":input(),  
    "age":input(),  
    "place":input()  
}  
print(student)
```

akhil
20
pulivendula
{'name': 'akhil', 'age': '20', 'place': 'pulivendula'}

```
student={  
    "name":"akhil",  
    "age":20,  
    "place":"pulivendula"
```

```
}  
print(student.get("name"),student.get("age"))
```

akhil 20

```
student["age"]=34  
print("after updating",student)
```

after updating {'name': 'akhil', 'age': 34, 'place': 'pulivendula'}

```
student['grade'] = 'A'  
print("After adding 'grade':", student)
```

After adding 'grade': {'name': 'akhil', 'age': 34, 'grade': 'A'}

```
del student['place']  
print("After deleting 'place':", student)
```

After deleting 'place': {'name': 'akhil', 'age': 20}

```
name = student.pop("name", None)  
print(name)  
print( student)
```

None
{'age': 20}

```
student={  
  
    "name":"akhil",  
  
    "age":20,  
  
    "place":"pulivendula"  
  
}  
print(student)
```

{'name': 'akhil', 'age': 20, 'place': 'pulivendula'}

```
student.keys()
```

```
dict_keys(['name', 'age', 'place'])
```

```
student.values()
```

```
dict_values([])
```

```
student.items()
```

```
dict_items([])
```

```
student.clear()
```

Start coding or [generate](#) with AI.

✓ sets operations in python

```
persons=(1,2,3,4,2,3,2,5,6,7,8,9)
print(set(persons))
```

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
persons.add(19)
print(persons)
```

```
{3, 4, 5, 6, 7, 8, 9, 19}
```

```
persons.discard(19)
print(persons)
```

```
{3, 4, 5, 6, 7, 8, 9}
```

```
persons.pop()
print(persons)
```

```
{6, 7, 8, 9}
```

```
persons.clear()
print(persons)
```

```
set()
```

union of sets

```
a={1,2,3,4,5}
b={4,5,6,7,8}
print(a|b)
```

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

```
a={1,2,3,4,5}
b={4,5,6,7,8}
print(a.union(b))
```

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

✓ set operations

```
a={1,2,3,4,5}
b={4,5,6,7,8}
print(a,b)
print(a|b)
print(a.union(b))
print(a- b)
print(a.difference(b))
print(a&b)
print(a.intersection(b))
print(a^b)
print(a.symmetric_difference(b))
```

```
{1, 2, 3, 4, 5} {4, 5, 6, 7, 8}
{1, 2, 3, 4, 5, 6, 7, 8}
{1, 2, 3, 4, 5, 6, 7, 8}
{1, 2, 3}
{1, 2, 3}
{4, 5}
{4, 5}
{1, 2, 3, 6, 7, 8}
{1, 2, 3, 6, 7, 8}
```

✓ functions in python

def is a key word to define a function

1. no parameters
2. with parameters

without parameters

```
def ksrn():
    print("my name is ksrn")
ksrn()
```

```
my name is ksrn
```

with parameters

```
def evenodd(a):
    if (a%2==0):
        print("even")
    else:
```

```
print("odd")
a=int(input("enter a number:"))
evenodd(a)
```

```
enter a number:5
odd
```

```
def evenodd(a):
    if (a%2==0):
        return "even number"
    else:
        return "odd number"
```

```
result=evenodd(a)
if (result=="even number"):
    print("working")
else:
    print("not working")
a=int(input("enter a number:"))
evenodd(a)
```

```
odd
not working
enter a number:2
even
```

✓ python dictionary practice

1. create a dictionary for a product with keys:name,price,brand,print each key-value pair

```
product = {
    "name": "Laptop",
    "price": 50000,
    "brand": "Dell"
}
print(product)
for key, value in product.items():
    print(key, ":", value)
```

```
{'name': 'Laptop', 'price': 50000, 'brand': 'Dell'}
name : Laptop
price : 50000
brand : Dell
```

2. Add a new key "in_stock" with value True. Print the updated dictionary.

```
product["in_stock"] = True

print(product)

{'name': 'Laptop', 'price': 55000, 'brand': 'Dell', 'in_stock': True}
```

3. Update the price to a new value and print the dictionary again.

```
product["price"] = 65000

print(product)

{'name': 'Laptop', 'price': 65000, 'brand': 'Dell', 'in_stock': True}
```

4. Ask the user to enter a student's name and marks in 3 subjects. Store in a dictionary and print.

```
student = {
    "name": input("Enter student name: "),
    "Subject1": int(input("Enter marks in Subject 1: ")),
    "Subject2": int(input("Enter marks in Subject 2: ")),
    "Subject3": int(input("Enter marks in Subject 3: "))
}
print(student)

Enter student name: akhil
Enter marks in Subject 1: 1
Enter marks in Subject 2: 2
Enter marks in Subject 3: 2
{'name': 'akhil', 'Subject1': 1, 'Subject2': 2, 'Subject3': 2}
```

5. Create a dictionary of 5 countries and their capitals. Print only the country names using keys().

```
countries = {
    "India": "New Delhi",
    "USA": "Washington D.C.",
    "Japan": "Tokyo",
    "France": "Paris",
    "Australia": "Canberra"
}

print(countries.keys())

dict_keys(['India', 'USA', 'Japan', 'France', 'Australia'])
```

6. Traverse a dictionary of 4 students and their scores. Print each name and score using items().

```
students = {  
    "Akhil": 90,  
    "harsha": 85,  
    "sravan": 95,  
    "vishnu": 88  
}  
  
for name, score in students.items():  
    print(name, ":", score)
```

```
Akhil : 90  
harsha : 85  
sravan : 95  
vishnu : 88
```

✓ python sets practice

1. create a set of 5 numbers with duplicates.print it and observe that duplicates are removed

```
numbers = {1, 2, 2, 3, 4}  
  
print(numbers)
```

```
{1, 2, 3, 4}
```

2. add number 100 to the set and print it.

```
numbers.add(100)  
print(numbers)
```

```
{1, 2, 3, 100, 4}
```

3. try to remove() on a value that is not in the set.then try discard().write down what happens

```
numbers.remove(10)  
print(numbers)
```

```
-----  
KeyError                                Traceback (most recent call last)  
/tmp/ipykernel_8323/2772623586.py in <cell line: 0>()  
----> 1 numbers.remove(10)  
      2 print(numbers)  
  
KeyError: 10
```



```
numbers.discard(10)
```

```
print(numbers)
```

```
{1, 2, 3, 100, 4}
```

4. create set A of subjects and set B of subjects your friend likes find subjects both of you likes

```
A = {"Maths", "Physics", "Python", "English"}
B = {"Python", "Biology", "Maths", "Chemistry"}
print(A&B)
```

```
{'Python', 'Maths'}
```

5. using same sets find all subjects either of you likes

```
print(A|B)
```

```
{'Physics', 'Biology', 'English', 'Chemistry', 'Python', 'Maths'}
```

6. convert this list to a set to remove duplicates:[1,2,2,3,4,4,5].print the result

```
numbers = [1, 2, 2, 3, 4, 4, 5]
```

```
result = set(numbers)
```

```
print(result)
```

```
{1, 2, 3, 4, 5}
```

✓ virtual judge problems

A - Print array

```
n = int(input())
a = list(map(int, input().split()))
for i in a:
    print(i, end=" ")
```

```
1
```

```
1 2 3 4 5 6
```

```
1 2 3 4 5 6
```

B - Summation

```
n = int(input())

a = list(map(int, input().split()))

print(sum(a))
```

```
2
1 2 3 5
11
```

C - Search with cops part 2

```
n = int(input())

a = list(map(int, input().split()))

x = int(input())

if x in a:
    print("Yes")
else:
    print("No")
```

```
5
1 2 3 4 5
5
Yes
```

D - Max one

```
n = int(input())

a = list(map(int, input().split()))

max_value = max(a)
index = a.index(max_value)

print(index)
```

```
4
1 2 3 4
3
```

E - MIN array

```
n = int(input())

a = list(map(int, input().split()))

min_value = a[0]
index = 0
```

```
for i in range(n):
    if a[i] < min_value:
        min_value = a[i]
        index = i

print(index)
```

```
4
1 2 3 4
0
```

F - Palindrome!

```
n = int(input())

a = list(map(int, input().split()))

palindrome = True

for i in range(n // 2):
    if a[i] != a[n - i - 1]:
        palindrome = False
        break

if palindrome:
    print("Yes")
else:
    print("No")
```

```
3
1 3 1
Yes
```

G - Local MAX

```
n = int(input())

a = list(map(int, input().split()))

count = 0

for i in range(1, n - 1):
    if a[i] > a[i - 1] and a[i] > a[i + 1]:
        count += 1

print(count)
```

```
4
1 2 3 4
0
```

